

4. Step 2 — Persistence base de données & ORM

Proposition de projet — Enterprise Inventory & Logistics Management System (EILMS)

Partie du document : Section 3.2 — Database Persistence & ORM Mapping

Objectif : passer du traitement en mémoire à une vraie persistance relationnelle d'entreprise.

Les 3 instructions du Step 2

1. Scripts JDBC natifs avec PreparedStatement : vérifier les métriques de pool de connexion directement en JDBC bas niveau. PreparedStatement (au lieu d'un simple Statement) précompile la requête SQL et protège contre les injections SQL. Exemple : `PreparedStatement ps = conn.prepareStatement("SELECT * FROM warehouse_items WHERE item_id = ?"); ps.setString(1, id);`

2. Entités JPA/Hibernate avec relations : @OneToMany et @ManyToOne pour modéliser les relations entre Warehouses, Items et Orders. Exemple : un Warehouse a plusieurs Items (@OneToMany côté Warehouse), et chaque Item appartient à un seul Warehouse (@ManyToOne côté Item). C'est la suite logique de l'entité WarehouseItemEntity déjà vue au Step 2 précédent, mais avec des relations entre plusieurs entités au lieu d'une seule table isolée.

3. Spring Data JPA Repositories avec JPQL : au lieu d'écrire l'EntityManager à la main (comme dans PersistenceTestMain.java), Spring Data JPA génère automatiquement l'implémentation d'une interface repository. Exemple : `interface ItemRepository extends JpaRepository<WarehouseItemEntity, String> { List<WarehouseItemEntity> findByQuantityLessThan(int seuil); }` — Spring génère automatiquement le SQL correspondant.

Lien avec ce qui a déjà été vu

Cette étape reprend directement les bases du document « Step 2 : Database Persistence Layer » déjà expliqué (JDBC, entité JPA, persistence.xml, EntityManager), mais y ajoute deux choses nouvelles : des relations entre plusieurs entités, et l'abstraction Spring Data JPA qui évite d'écrire le code EntityManager à la main.